Student Keystore Setup Guide

app-bff — Spring Boot API Gateway with Native JWT Auth & JDBC DAL

Author: R. Seeds

Status: Verified against live project source as of 2026-08-28

Student Keystore Setup Guide

Creating Your Local HTTPS Keystore (mkcert) for app-bff

Why You're Doing This

The app-bff server runs over HTTPS on port 8443. To do that, it needs a TLS certificate — a keystore.p12 file — sitting in src/main/resources. That file **is not something you can copy from a classmate or the instructor's machine and expect to just work smoothly**: the certificate is signed by a locally-generated trust authority (mkcert's local CA) that only your machine trusts once you install it. Everyone generates their own.

The good news: it's four commands and about two minutes.

Quick Checklist

- ☐ Install mkcert
- ☐ Trust the local CA (mkcert -install)
- ☐ Generate keystore.p12
- ☐ Re-key it to the class-standard password
- ☐ Verify the password and the alias
- ☐ Copy it into src/main/resources/
- ☐ Update application.properties to match
- ☐ Rebuild and test

Step 1: Install mkcert

Pick the command for your OS:

Windows (winget — recommended):

```
winget install FiloSottile.mkcert
```

Windows (Chocolatey / Scoop, alternatives):

```
choco install mkcert
# OR
scoop install mkcert
```

macOS (Homebrew):

```
brew install mkcert
brew install nss   # optional: enables Firefox support
```

Linux (Ubuntu/Debian):

```
sudo apt install libnss3-tools
```

```
# then download the mkcert binary from GitHub releases and:
sudo cp mkcert-v*-linux-amd64 /usr/local/bin/mkcert
sudo chmod +x /usr/local/bin/mkcert
```

> **Gotcha**
>
> if you installed mkcert and immediately try to run it in the same terminal window you had open during the install, it may not be found yet:
>
> **Windows (winget/choco/scoop):** `mkcert : The term 'mkcert' is not recognized...`
>
> **macOS/Linux (brew/apt):** `zsh: command not found: mkcert` or `bash: mkcert: command not found`
>
> Either way, it's a stale PATH in that terminal session — the install updated your PATH, but the terminal you already had open loaded its environment before that happened. Close it and open a fresh terminal (or a new tab/panel in your IDE) and it'll resolve.
>
> **Apple Silicon Mac note:** if mkcert still isn't found even in a brand-new terminal after brew install mkcert, your shell profile may not have Homebrew's /opt/homebrew/bin on PATH yet (common on a freshly-set-up Mac). Run `eval "$(/opt/homebrew/bin/brew shellenv)"` once, or fully quit and reopen your terminal app, and it should resolve.

Step 2: Trust the Local Certificate Authority

```
mkcert -install
```

This installs a certificate authority unique to your machine into your OS (and browser, where applicable) trust store. This is the step that makes your locally-generated certificate show up as "trusted" instead of throwing browser warnings — but only on your machine. That's expected and fine; it's not meant to be shared.

Step 3: Generate Your Keystore

Pick a working folder you'll remember — for example, your project's root folder — and run:

```
mkcert -pkcs12 -p12-file keystore.p12 localhost 127.0.0.1 ::1
```

> **Gotcha**
>
> mkcert writes the output file to whatever directory your terminal is currently sitting in. If you run this from your home directory (~) or some other folder you weren't paying attention to, the file will land there instead of somewhere useful, and it's easy to lose track of in a long scrollback. Know which folder you're in before you run the command — pwd (bash) or Get-Location (PowerShell) if you're not sure.

You'll see output like this:

```
Note: the local CA is not installed in the Java trust store.
Run "mkcert -install" for certificates to be trusted automatically
```

```
Created a new certificate valid for the following names
 - "localhost"
 - "127.0.0.1"
 - "::1"

The PKCS#12 bundle is at "keystore.p12"

The legacy PKCS#12 encryption password is the often hardcoded default "changeit"
```

That last line is important — read Step 4 before doing anything else.

Step 4: Set the Password — Read This Carefully

This is the step almost everyone trips on, and it's worth slowing down for.

mkcert always uses "changeit" — no exceptions

You might see instructions elsewhere (or be tempted to try) setting an environment variable like $env:KEYSTORE_PASSWORD before running the mkcert command, expecting it to bake in a custom password. **It doesn't work.** mkcert has no flag or environment variable that changes the PKCS12 password — every file it generates is password-protected with the literal string changeit, no matter what you do beforehand. The tool tells you this itself in its own output (see the "hardcoded default" line above).

Re-key it to the class-standard password

> **IMPORTANT**
>
> **Use the same password everyone else in the cohort is using:** `#FSISeedsSWDJune2026`. Don't invent your own.

Here's why this matters more than it might seem:

- The reference application.properties you were given already has server.ssl.key-store-password=#FSISeedsSWDJune2026 in it. If your keystore's actual password doesn't match that value exactly, the app fails at startup with a cryptic BadPaddingException / UnrecoverableKeyException — a real, confusing error that has nothing obviously to do with "wrong password" unless you already know to look there.
- If everyone in the cohort uses the same password, troubleshooting help from instructors/TAs/classmates is dramatically faster — nobody has to first ask "what password did you use?" before they can help you debug anything else.
- A mismatched or forgotten custom password is the single most common way this setup breaks. Don't be clever here — use the provided one.

Re-key your freshly-generated file with keytool (this ships with your JDK):

```
keytool -storepasswd -new "#FSISeedsSWDJune2026" -keystore keystore.p12 -storetype PKCS12 -storepass changeit
```

Read that command carefully:

- -storepass changeit is the current password (what mkcert just set — always changeit, every time, for every student).
- -new "#FSISeedsSWDJune2026" is the password you're changing it to.

> **Careful with the # character**
>
> `#FSISeedsSWDJune2026` starts with a #. In bash and zsh (the default shells on macOS/Linux) and in PowerShell (Windows), # doesn't have special meaning inside double quotes — the command above is safe to copy-paste exactly as written on any OS. The real danger is typing it unquoted: at a bare bash/zsh prompt, an unquoted # starts a comment and silently truncates everything after it — so if you ever paste this password outside of quotes (into a shell command, a .env file, anywhere), double-check it didn't get chopped off. When in doubt, keep it in quotes, and confirm what actually got saved with the verification command below — don't just trust that it "probably worked."

Verify it actually took

Don't just trust that the command succeeded silently — confirm it:

```
keytool -list -v -keystore keystore.p12 -storetype PKCS12 -storepass "#FSISeedsSWDJune2026"
```

You should see output including:

```
Entry type: PrivateKeyEntry
```

If that command fails (wrong password error) or doesn't show PrivateKeyEntry, stop here and re-run the re-key command before moving on — everything past this point will fail in confusing ways if the password is wrong, and it's much easier to fix now than after you've moved the file into your project and forgotten which password you actually used.

Step 5: Check the Alias

```
keytool -list -keystore keystore.p12 -storetype PKCS12 -storepass "#FSISeedsSWDJune2026"
```

Look for a line like:

```
1, <date>, PrivateKeyEntry,
```

That 1 at the start of the line is the alias — mkcert always names it 1, regardless of hostname, machine, or who's running it. You don't need to change anything here; the reference application.properties already expects server.ssl.key-alias=1. This is just worth seeing once so it's not a surprise later.

Step 6: Add It to Your Project

Copy (don't just leave it wherever you generated it) your finished, re-keyed keystore.p12 into:

```
app-bff/src/main/resources/keystore.p12
```

This exact location matters — application.properties references it as classpath:keystore.p12, which means Spring Boot looks for it on the compiled classpath, which is built from src/main/resources.

> **Gotcha**
> if a keystore.p12 already exists at that path (e.g. a placeholder from the project template), overwrite it with yours. And after copying it in, trigger a real rebuild — mvn clean compile, or Build → Rebuild Project in IntelliJ — before running the app. Maven copies src/main/resources into target/classes as a build step; if you skip the rebuild, the app can keep loading an old, stale copy and it'll look like your fix "didn't work" when really it just never got picked up.

Step 7: Confirm application.properties Matches

Your SSL block should read exactly:

```
server.port=8443
server.ssl.enabled=true
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=#FSISeedsSWDJune2026
server.ssl.key-alias=1
```

If you followed Steps 4–5 as written (used the class-standard password, didn't touch the alias), this should already match what you were given — you shouldn't need to change this file at all. If it doesn't match, that's your signal something upstream (the password you actually set, most likely) drifted from what's expected.

Step 8: Rebuild and Test

- Rebuild the project (see the gotcha in Step 6 if you haven't already).
- Run AppBffApplication (via IntelliJ's run configuration, or mvn spring-boot:run).
- A clean startup shows the Spring Boot banner and no stack trace, serving on https://localhost:8443.
- Quick functional check:

```
curl -sk -i -X POST https://localhost:8443/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"testuser","password":"testpass"}'
```

Expect 201 Created with the body "User registered successfully". That confirms your keystore loaded correctly and the server is actually serving HTTPS traffic — the register endpoint itself doesn't touch TLS logic beyond "the server started at all," so a successful response here is a solid green light for this whole setup.

If Something Goes Wrong

Symptom	Likely Cause	Fix
UnrecoverableKeyException / BadPaddingException on startup	Your keystore's real password doesn't match server.ssl.key-store-password in application.properties	Re-verify with keytool -list -v ... -storepass "#FSISeedsSWDJune2026" (Step 4). If that fails, regenerate the keystore (Step 3) and redo Step 4 carefully.
Same error, but you're sure the password is right	You edited/replaced keystore.p12 but never rebuilt, so target/classes still has the old file	Rebuild: mvn clean compile
Startup can't find a key aliased something other than 1	server.ssl.key-alias was changed away from 1	Set it back to server.ssl.key-alias=1 — mkcert's alias is always 1
mkcert command not found	Terminal session predates the install	Open a new terminal
Can't find keystore.p12 after running mkcert	It landed in whatever directory your terminal was in when you ran the command	Check your terminal's prompt for the folder you were in, or search for the file and move it

If none of these match what you're seeing, don't guess — bring the exact error message (the full stack trace, not just the first line) to your instructor or a TA. The real cause is almost always visible a few lines down in a "Caused by:" chain, even when the top-level error looks unrelated to certificates.